Desarrollo Web
Despliegue de Backend

Django · Python · PostgreSQL · Render

Junior Perez Davila · 407721

Claudia Libertad Quispe Terrones · 407719

Samuel Andres Zona Malaver · 385387

Proyecto: mi_proyecto_routing

Repositorio: https://github.com/JuniorPD04/practica_django

PENDIENTE: desplegar y regenerar este PDF con --url antes de entregar.

Fecha de preparación: 13/11/2026

Cuestionario: despliegue de backend Django

1. ¿Qué significa desplegar una aplicación web?

Es publicar la aplicación en un servidor para que otras personas accedan mediante una URL. Implica subir el código, instalar dependencias, configurar secretos y dominios, conectar la base de datos, aplicar migraciones e iniciar el servidor. Después se comprueba que las rutas respondan correctamente desde internet y se revisan los registros si hay errores.

2. ¿Qué es Render? Ventajas y limitaciones

Render es una plataforma para alojar aplicaciones y bases de datos administradas. Simplifica este taller porque integra despliegues desde GitHub y HTTPS automático. También ofrece instancias gratuitas útiles para una práctica. Sus límites incluyen la suspensión del servicio web tras 15 minutos sin tráfico y la expiración del PostgreSQL gratuito a los 30 días. Además, el disco del servicio web es efímero: los datos persistentes deben guardarse en PostgreSQL. Fuente: https://render.com/docs/free

3. Diferencias entre desarrollo y producción

En local se usa runserver, mientras que en producción se usa Gunicorn. Localmente puede usarse SQLite; el despliegue usa PostgreSQL. DEBUG se activa para depurar y se desactiva al publicar. Los secretos y hosts cambian entre entornos y se configuran externamente. Además, producción requiere HTTPS, cookies seguras, gestión de estáticos y revisión de logs; los errores no deben mostrar trazas privadas a los usuarios.

4. Variables de entorno y credenciales

Son valores que recibe el proceso desde su entorno, como SECRET_KEY o DATABASE_URL. Separan configuración y código. Escribir contraseñas en el código puede publicarlas en Git y dificulta cambiarlas entre entornos. En desarrollo usamos un .env ignorado por Git; en Render se configuran en Environment o mediante el Blueprint. Las variables del sistema tienen prioridad sobre el archivo local. El Blueprint genera una SECRET_KEY nueva y enlaza la base de datos sin publicar su contraseña.

5. Pasos para cambiar de motor y relación con el ORM

Se añadió psycopg2-binary como controlador, dj-database-url para interpretar la conexión y una configuración DATABASES que prioriza DATABASE_URL. En Render se crea PostgreSQL y se ejecuta migrate sobre la base nueva. No hay que reescribir las consultas porque los modelos y QuerySets utilizan el ORM. Los datos de SQLite no se transfieren automáticamente: el proyecto ofrece cargar un fixture en una base vacía; para conservar otros datos se necesita una migración de datos aparte.

6. ¿Qué es PaaS? Comparación

PaaS ofrece una plataforma que administra infraestructura y ejecución para que el equipo se concentre en su aplicación. Render aloja un proceso web Django y permite conectarlo a PostgreSQL administrado. Vercel también abstrae infraestructura y está especialmente orientado a frontends y funciones; ejecutar Django allí requiere adaptar el proyecto a su modelo y mantener una base externa persistente. Ambas reducen administración, pero sus modelos de ejecución y límites deben evaluarse para cada aplicación. Fuentes: https://render.com/docs y

https://vercel.com/docs/frameworks/backend/django

7. ¿Por qué se obtiene HTTPS automáticamente?

Render administra los certificados TLS y recibe las conexiones HTTPS en su infraestructura. Después reenvía las solicitudes al proceso Django. Configuramos SECURE_PROXY_SSL_HEADER para que Django reconozca el protocolo original informado por el proxy y no genere redirecciones incorrectas. No necesitamos instalar manualmente un certificado para el dominio onrender.com. Fuente: https://render.com/docs/tls

8. Cold start

Después de 15 minutos sin tráfico, Render suspende el Web Service gratuito. La siguiente solicitud reactiva el servicio y puede tardar aproximadamente un minuto. En una demo conviene abrir /authors con anticipación; un usuario real puede experimentar espera en su primera visita. La demora no significa necesariamente que el código haya fallado. Fuente: https://render.com/docs/free

9. Alternativas para conservar la base sin costo

Neon ofrece un plan gratuito de PostgreSQL sin un plazo fijo de 30 días, sujeto a sus cuotas de almacenamiento y cómputo. Supabase también ofrece PostgreSQL gratuito con límites de uso; sus proyectos Free pueden pausarse tras una semana de inactividad. Para cambiar se crea la base, se copian los datos antes de que expire la anterior, se configura la conexión con TLS que exige el proveedor y se actualiza DATABASE_URL tanto en Render como en la declaración del Blueprint. Fuentes: https://neon.com/pricing y https://supabase.com/pricing

10. Seguridad básica al publicar la API

Desactivar DEBUG y usar SECRET_KEY privada; restringir ALLOWED_HOSTS; servir por HTTPS y usar cookies seguras; validar entradas; y aplicar autenticación y permisos a las rutas que lo requieran. También deben evitarse contraseñas o tokens en logs y mantenerse las dependencias actualizadas. La configuración global DRF no protege automáticamente vistas Django convencionales: en este proyecto existen rutas históricas públicas del juego, así que un sistema con datos privados necesitaría definir permisos también para ellas. /authors es público por requisito.

11. ¿Por qué Gunicorn en lugar de runserver?

runserver sirve para desarrollar y depurar, no para gestionar tráfico de producción. Gunicorn es un servidor WSGI que administra procesos de trabajo, recibe solicitudes y llama a la aplicación Django. Render ejecuta Gunicorn en Linux y le asigna un puerto mediante PORT. En Windows seguimos usando runserver para desarrollo. Fuente: https://docs.djangoproject.com/en/6.1/howto/deployment/wsgi/gunicorn/

12. Función de render.yaml

Declara los servicios, planes, región, comandos de build y arranque y variables de entorno. En este repositorio identifica expresamente mi_proyecto_routing como raíz del Web Service y enlaza routing-db mediante fromDatabase. Su ventaja es que la configuración puede revisarse y versionarse junto con el código, lo que reduce diferencias por clics manuales. Los valores secretos se generan o se proporcionan desde Render. Fuente: https://render.com/docs/blueprint-spec

13. Cambios concretos en settings.py

SECRET_KEY y DEBUG pasan a leerse desde el entorno. ALLOWED_HOSTS incluye localhost y el hostname exacto asignado por Render. DATABASES interpreta DATABASE_URL para PostgreSQL y conserva SQLite local; en Render exige una conexión configurada. STATIC_ROOT concentra los archivos recogidos por collectstatic. STORAGES usa el almacenamiento comprimido con manifiesto de WhiteNoise y su middleware va después de SecurityMiddleware. También se registra rest_framework y se configura el protocolo del proxy, redirección HTTPS y cookies seguras. Se utiliza STORAGES, que es la configuración vigente, en lugar del antiguo STATICFILES_STORAGE.

14. Propósito de WhiteNoise

Con DEBUG=False, Django no sirve automáticamente los estáticos como durante el desarrollo. collectstatic los reúne en STATIC_ROOT y WhiteNoise permite entregarlos desde la aplicación desplegada, sin instalar otro servidor de archivos. Su almacenamiento comprime recursos y genera nombres con huella para caché. Esto permite que el administrador y los recursos del proyecto carguen correctamente. No sustituye un almacenamiento persistente para archivos que suban los usuarios. Fuente: https://whitenoise.readthedocs.io/en/stable/django.html

15. Modelos y migraciones al cambiar a PostgreSQL

Los modelos describen entidades y relaciones mediante tipos de Django, y el ORM convierte operaciones Python al SQL correspondiente. Las migraciones existentes usan esas operaciones abstractas, por eso pueden crear el esquema también en PostgreSQL. No se modificaron modelos ni migraciones para este despliegue. Esa portabilidad tiene límites si se usan consultas SQL o funciones específicas de un motor; además, crear el esquema no copia los registros que ya existían en SQLite. Es necesario verificar el funcionamiento sobre PostgreSQL cuando el servicio esté desplegado.